

Семинар №12

SFINAE, requires

SFINAE

- SFINAE – substitution failure is not an error
- Если ошибка при подстановке типа в шаблонный параметр происходит при выборе перегрузки – то она не приводит к ошибке компиляции. Просто эта версия перегрузки не выбирается.
- Эта ошибка может быть в трех местах:
 - в типе возвращаемого значения
 - в типе аргумента функции
 - в шаблонных параметрах
- Но **НЕ** может быть в теле функции!

Общая идея

- Делаем одну более предпочтительную с точки зрения аргументов версию, и одну менее (например, принимающую '...')
- В более предпочтительную версию вставляем SFINAE

Реализация `is_class`

- Наследуемся от типа, равного возвращаемому значению функции: `decltype(detail::test<T>(nullptr))`
- Если `nullptr` можно скастить к указателю на метод класса, то `T` это класс (только у классов могут быть указатели на методы)
- Причем не важно, есть ли поле такого типа. Сам тип `int T::*` будет всегда корректен.
- Если скастить к такому типу нельзя, то это не класс

Comma trick

- Иногда хотим проверить, корректна ли шаблонная подстановка для некоторого типа, но не хотим использовать получившийся тип
- Используем оператор “,” , например:
 - `std::true_type helper(decltype(T(std::declval<Args>())...), nullptr);`

Реализация `is_polymorphic`

- Идея как в `is_class`
- Нужно понять, как отличить в compile time полиморфный тип от неполиморфного.
- Можем использовать `dynamic_cast`: `dynamic_cast` неполиморфного типа к `void*` приводит к СЕ

Функция declval

- Функция для метапрограммирования, может быть использована только в unevaluated context
- Нужна только для того, чтобы “конструировать” объект типа T в метафункциях (помогает избегать необходимости в конструкторах по умолчанию):

```
template <typename T>  
T&& declval() { static_assert(false); }
```

- T&& вместо T потому, что иначе нельзя было бы использовать с incomplete types, а также пришлось бы всегда инстанцировать шаблоны

Common Types

- Хотим вывести наименьшего общего предка для набора типов
- Используем тернарный оператор

SFINAE-friendly

- Не все функции являются SFINAE-friendly, то есть могут использоваться в контексте `sfinae`
- Например, если возвращаемое значение функции невозможно понять только по её сигнатуре и нужно инстанцировать всю функцию

Requires

- Хотим более удобный синтаксис для ограничений на тип шаблонного аргумента
- Requires: после шаблонных аргументов (или после сигнатуры перед скобками {) можно написать требования:
 `requires *compile time bool value*`
- Можно делать функции, которые отличаются только `requires`
- Но перегрузка не умеет упорядочивает `requirements` по строгости, только по наличию / отсутствию
- Могут быть только на шаблонных функциях

Require Expressions

- `requires ( parameter-list(optional) ) { requirement-seq }`
- Превращается в булево выражение в компайл-тайме
- Можно писать любые expression
- Проверяется только компилируемость выражений, не истинность
- Виды `requires expression`:
 - Simple requirement
 - Nested requirement
 - Type requirement
 - Compound requirement